Clasificación lineal y regresión logística

Tomas Santana
October, 2026

Índice

1. Parte 1: Clasificación lineal 1
   1.1. ¿Por qué no usar regresión lineal? 1
   1.2. Score, regla de decisión y frontera 2
   1.3. Geometría de la frontera 2
2. Parte 2: Regresión logística 3
   2.1. La función sigmoide 3
   2.2. Un modelo de probabilidad para y 4
3. Parte 3: Máxima verosimilitud 5
   3.1. Verosimilitud y log-verosimilitud 5
   3.2. Lo que aporta cada ejemplo 6
4. Parte 4: Entrenamiento 6
   4.1. El gradiente 6
   4.2. Ascenso por gradiente 7
   4.3. Datos separables y regularización 7
5. Parte 5: Transformaciones de features 8
   5.1. Fronteras no lineales con un modelo lineal 8
   5.2. El costo de muchas features 9
   5.3. Hacia SVM y kernels 10
6. Resumen 11

Hasta ahora hemos trabajado con problemas de regresión, donde la etiqueta y es un número real. En estas notas pasamos a problemas de clasificación binaria, donde y solo puede tomar dos valores, que representamos como $y \in \{0, 1\}$. Por ejemplo, $y = 1$ si un correo es spam y $y = 0$ si no lo es. A la clase $y = 1$ se le llama clase positiva y a $y = 0$ clase negativa.

La idea central es que podemos reutilizar casi todo lo que ya sabemos de regresión lineal: el modelo sigue usando $\theta^T \mathbf{x}$, los parámetros se escogen por máxima verosimilitud, y se entrenan con un método de gradiente. Lo que cambia es la forma en que el modelo describe la probabilidad de y.

Seguimos la notación de regresión lineal: m ejemplos de entrenamiento $\left(\mathbf{x}^{(i)}, y^{(i)}\right)$, parámetros $\theta_0, ..., \theta_{n-1}$, y la convención $x_0 = 1$, de forma que $\theta^T \mathbf{x} = \theta_0 + \theta_1 x_1 + ... + \theta_{n-1} x_{n-1}$.

Estas notas siguen los capítulos 2 y 5 de las notas de Stanford CS229 y la sección 4.3 de ISLP.

1. Parte 1: Clasificación lineal

1.1. ¿Por qué no usar regresión lineal?

La primera idea sería ignorar que y solo toma los valores 0 y 1, ajustar una regresión lineal como siempre, y predecir $y = 1$ cuando $h_\theta(\mathbf{x}) \geq 0.5$. En algunos casos esto funciona, pero es fácil construir ejemplos donde falla.

En la Figura 1, x es el tamaño de un tumor y $y = 1$ indica que es maligno. A la izquierda, la recta de mínimos cuadrados cruza 0.5 en $x \approx 4.63$, que separa bien las dos clases. A la derecha agregamos tres tumores muy grandes, todos malignos. Estos ejemplos no deberían cambiar nada: son casos

fáciles. Pero la recta se inclina para acercarse a ellos, el umbral se mueve a $x \approx 5.7$, y ahora 2 tumores malignos quedan mal clasificados.

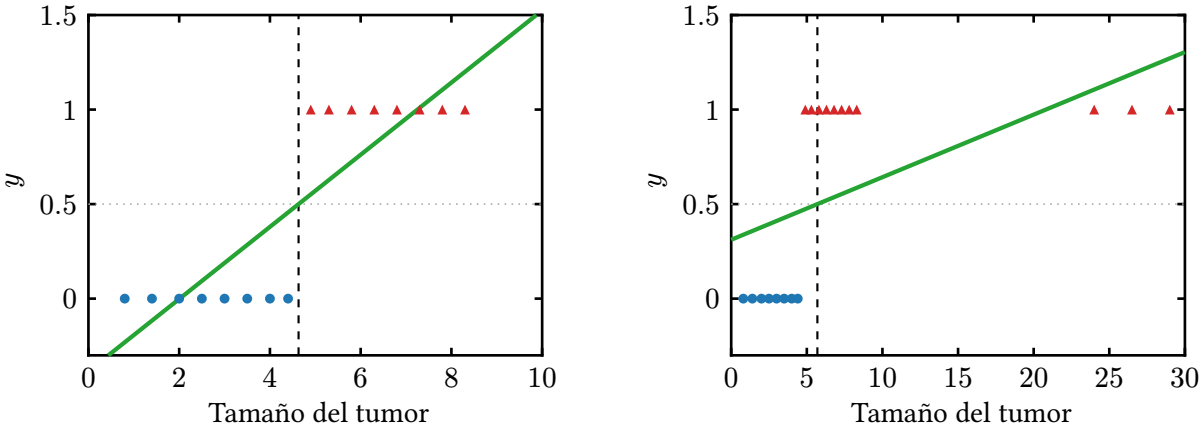


Figura 1: Regresión lineal aplicada a etiquetas 0/1. La recta verde es el ajuste de mínimos cuadrados y la línea punteada vertical es el punto donde la recta vale 0.5. Al agregar tres ejemplos fáciles a la derecha, la recta cambia de pendiente y el umbral se desplaza, clasificando mal ejemplos que antes estaban bien.

Hay dos problemas de fondo. El primero es que el error cuadrático castiga a la recta por predecir valores lejanos a 1 para los tumores grandes, aunque esas predicciones estén del lado correcto. El segundo es que $h_\theta(\mathbf{x})$ puede tomar cualquier valor real, mientras que sabemos que $y \in \{0, 1\}$. Una predicción de 3.7 o de -2 no tiene una interpretación clara.

Vamos a resolver ambos problemas, pero primero conviene entender qué significa clasificar con un modelo lineal.

1.2. Score, regla de decisión y frontera

Llamamos score al valor $z = \theta^T \mathbf{x}$. Un clasificador lineal predice usando el signo del score:

$$\hat{y} = \begin{cases} 1 & \text{si } \theta^T \mathbf{x} \geq 0 \\ 0 & \text{si } \theta^T \mathbf{x} < 0. \end{cases}$$

Los puntos donde el score vale exactamente cero forman la frontera de decisión:

$$\{\mathbf{x} : \theta^T \mathbf{x} = 0\}.$$

Con dos variables de entrada, la frontera es $\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$, que es una recta. Con tres variables es un plano, y en general es un hiperplano. La frontera divide el espacio en dos mitades: de un lado el modelo predice la clase 1 y del otro la clase 0. Por eso se dice que el clasificador es lineal: no porque el modelo sea una recta, sino porque la frontera lo es.

Si existe algún θ cuya frontera deja todos los ejemplos de entrenamiento del lado correcto, decimos que los datos son linealmente separables.

1.3. Geometría de la frontera

Tomemos el caso de dos variables. El vector (θ_1, θ_2) es perpendicular a la frontera. Para verlo, tomemos dos puntos $\mathbf{a}$ y $\mathbf{b}$ sobre la frontera. Restando sus ecuaciones:

$$(\theta_0 + \theta_1 a_1 + \theta_2 a_2) - (\theta_0 + \theta_1 b_1 + \theta_2 b_2) = \theta_1(a_1 - b_1) + \theta_2(a_2 - b_2) = 0.$$

Es decir, (θ_1, θ_2) es perpendicular a $\mathbf{a} - \mathbf{b}$, que es un vector en la dirección de la frontera. A (θ_1, θ_2) se le llama vector normal. Además, apunta hacia el lado donde el score es positivo, es decir, hacia la

región que el modelo clasifica como 1. El parámetro θ_0 no cambia la orientación de la frontera: solo la desplaza.

El score también dice qué tan lejos está un punto de la frontera. Se puede demostrar que la distancia (con signo) de un punto $\mathbf{x}$ a la frontera es

$$\frac{\theta_0 + \theta_1 x_1 + \theta_2 x_2}{\sqrt{\theta_1^2 + \theta_2^2}}.$$

Así, un score grande y positivo significa que el punto está lejos de la frontera, del lado de la clase 1. Esto lo vamos a interpretar como que el modelo está muy seguro de su predicción.

Un detalle importante: si multiplicamos todos los parámetros por una constante $c > 0$, la frontera no cambia, porque $c\theta^T\mathbf{x} = 0$ si y solo si $\theta^T\mathbf{x} = 0$. Pero los scores sí cambian: se multiplican por c. Volveremos a esto al entrenar.

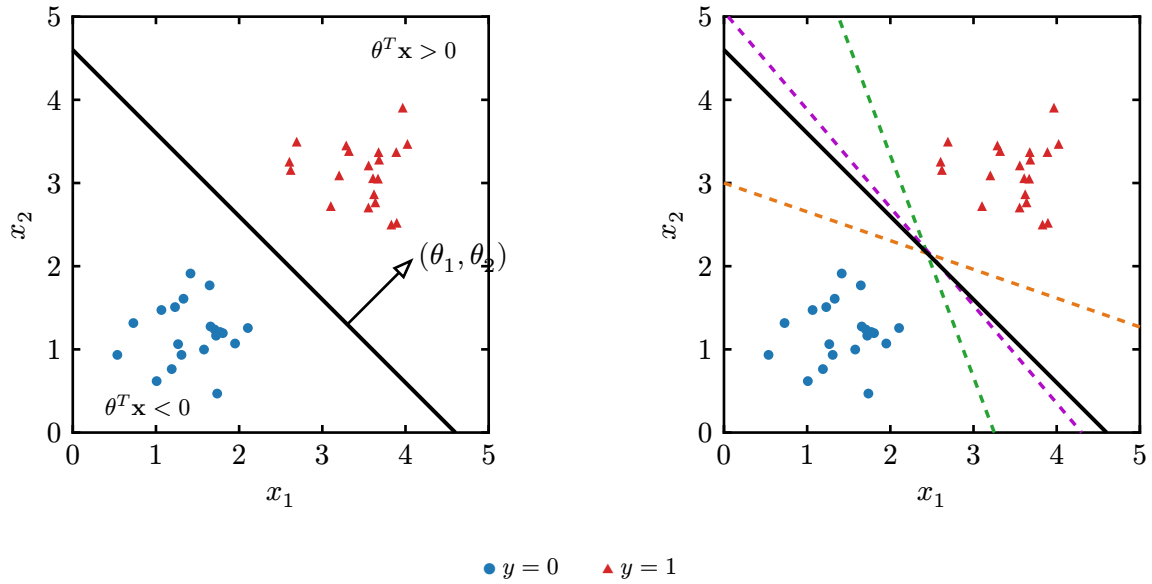


Figura 2: A la izquierda, una frontera lineal $\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$ con su vector normal (θ_1, θ_2), que apunta hacia la región donde el modelo predice $y = 1$. A la derecha, los mismos datos con otras tres fronteras que también separan perfectamente las dos clases.

La Figura 2 deja abierta una pregunta: cuando los datos son separables, hay infinitas fronteras que clasifican perfectamente el conjunto de entrenamiento. ¿Cuál deberíamos escoger? Intuitivamente, una frontera que pasa muy cerca de algunos puntos parece más frágil que una que deja espacio a ambos lados. Esta idea dará origen a las máquinas de soporte vectorial (SVM). Por ahora, vamos a escoger θ con un criterio probabilístico.

2. Parte 2: Regresión logística

2.1. La función sigmoide

En lugar de predecir directamente 0 o 1, nos gustaría que el modelo diera una probabilidad: un número entre 0 y 1 que indique qué tan probable es que $y = 1$. Para eso tomamos el score $\theta^T\mathbf{x}$, que puede ser cualquier número real, y lo pasamos por una función que lo comprima al intervalo $(0, 1)$:

$$h_\theta(\mathbf{x}) = g(\theta^T\mathbf{x}) = \frac{1}{1 + e^{-\theta^T\mathbf{x}}},$$

donde

$$g(z) = \frac{1}{1 + e^{-z}}$$

se llama función logística o función sigmoide. A este modelo se le llama regresión logística. A pesar del nombre, es un método de clasificación: la palabra "regresión" viene de que el modelo predice un número (la probabilidad).

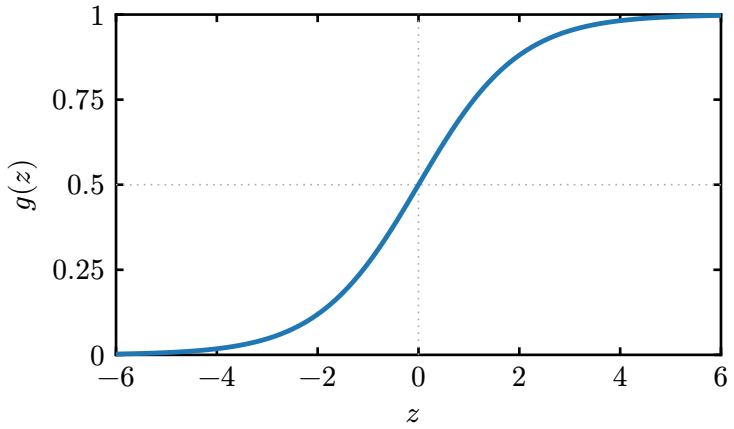


Figura 3: La función sigmoide $g(z) = 1/(1 + e^{-z})$. Cuando el score z es muy positivo, $g(z)$ se acerca a 1; cuando es muy negativo, se acerca a 0. Cuando $z = 0$, $g(z) = 0.5$.

Algunas propiedades de la sigmoide que usaremos:

- $g(z) \to 1$ cuando $z \to \infty$, y $g(z) \to 0$ cuando $z \to -\infty$.
- $g(0) = 0.5$.
- $g(-z) = 1 - g(z)$. Es decir, la curva es simétrica alrededor del punto $(0, 0.5)$.
- Su derivada se puede escribir en términos de la misma función:

$$g'(z) = \frac{d}{dz}\frac{1}{1 + e^{-z}} = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \left(1 - \frac{1}{1 + e^{-z}}\right) = g(z)(1 - g(z)).$$

En el tercer paso usamos que $e^{-z}/(1 + e^{-z}) = 1 - 1/(1 + e^{-z})$.

La sigmoide no es la única función que va suavemente de 0 a 1, pero es la más usada, y tiene propiedades que simplifican mucho las cuentas.

2.2. Un modelo de probabilidad para y

Interpretamos la salida del modelo como la probabilidad de que $y = 1$ dado $\mathbf{x}$:

$$P(y = 1 \mid \mathbf{x}; \theta) = h_\theta(\mathbf{x}), \qquad P(y = 0 \mid \mathbf{x}; \theta) = 1 - h_\theta(\mathbf{x}).$$

Las dos ecuaciones se pueden escribir en una sola:

$$p(y \mid \mathbf{x}; \theta) = (h_\theta(\mathbf{x}))^y (1 - h_\theta(\mathbf{x}))^{1-y}.$$

Basta sustituir $y = 1$ y $y = 0$ para comprobarlo: en cada caso, uno de los dos factores se vuelve 1.

Para clasificar, predecimos $y = 1$ cuando $h_\theta(\mathbf{x}) \geq 0.5$. Como $g(z) \geq 0.5$ exactamente cuando $z \geq 0$, esto es lo mismo que predecir $y = 1$ cuando $\theta^T \mathbf{x} \geq 0$. Es decir, la regresión logística es un clasificador lineal: su frontera de decisión es el mismo hiperplano $\theta^T \mathbf{x} = 0$ de la Parte 1. Lo que agrega es una probabilidad para cada punto, que depende de qué tan lejos está de la frontera.

Más en general, los puntos donde el modelo asigna una probabilidad fija p cumplen $g(\theta^T \mathbf{x}) = p$, es decir, $\theta^T \mathbf{x} = \log(p/(1 - p))$. Son rectas paralelas a la frontera, como se ve en la Figura 4.

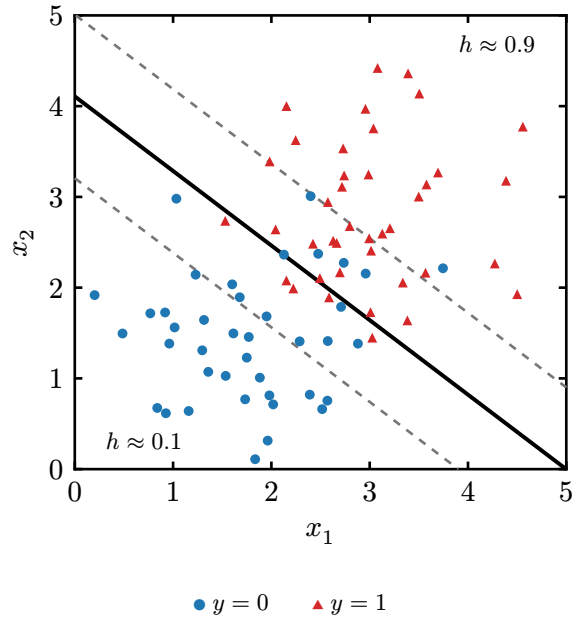


Figura 4: Regresión logística ajustada a 80 ejemplos cuyas clases se solapan. La recta continua es la frontera de decisión ($h_\theta(\mathbf{x}) = 0.5$), y las rectas punteadas son los puntos donde el modelo asigna probabilidad 0.1 y 0.9 a la clase 1. Todas son paralelas. El modelo clasifica bien el 86% de los ejemplos de entrenamiento.

Hay otra forma útil de leer el modelo. Si despejamos el score:

$$\frac{h_\theta(\mathbf{x})}{1 - h_\theta(\mathbf{x})} = e^{\theta^T \mathbf{x}} \quad \text{y por lo tanto} \quad \log \frac{h_\theta(\mathbf{x})}{1 - h_\theta(\mathbf{x})} = \theta^T \mathbf{x}.$$

Al cociente $p/(1-p)$ se le llama odds (por ejemplo, $p = 0.75$ corresponde a odds de 3 a 1), y a su logaritmo se le llama log-odds o logit. Entonces la regresión logística supone que el log-odds de la clase 1 es una función lineal de $\mathbf{x}$. Esto da una interpretación de los coeficientes: si x_j aumenta en una unidad y lo demás queda igual, el log-odds aumenta en θ_j, o sea, el odds se multiplica por e^{θ_j}.

3. Parte 3: Máxima verosimilitud

Ya tenemos el modelo. Falta escoger θ. Para la regresión lineal vimos que, si suponemos $y = \theta^T \mathbf{x} + \varepsilon$ con $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, maximizar la verosimilitud es lo mismo que minimizar el error cuadrático. Vamos a seguir la misma receta: escribir la probabilidad que el modelo le asigna a los datos y escoger el θ que la hace más grande.

3.1. Verosimilitud y log-verosimilitud

Suponemos que los m ejemplos de entrenamiento se generaron de forma independiente. Entonces la verosimilitud de los parámetros es el producto de las probabilidades de cada etiqueta:

$$L(\theta) = p(\mathbf{y} \mid \mathbf{X}; \theta) = \prod_{i=1}^{m} p\big(y^{(i)} \mid \mathbf{x}^{(i)}; \theta\big) = \prod_{i=1}^{m} \big(h_\theta(\mathbf{x}^{(i)})\big)^{y^{(i)}} \big(1 - h_\theta(\mathbf{x}^{(i)})\big)^{1 - y^{(i)}}.$$

$L(\theta)$ es la probabilidad que el modelo le asigna a las etiquetas que efectivamente observamos. El principio de máxima verosimilitud dice que debemos escoger el θ que la hace más grande.

Igual que en regresión lineal, es más cómodo maximizar el logaritmo, que convierte el producto en una suma:

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^{m} y^{(i)} \log h_\theta\big(\mathbf{x}^{(i)}\big) + \big(1 - y^{(i)}\big) \log\big(1 - h_\theta\big(\mathbf{x}^{(i)}\big)\big).$$

Como el logaritmo es una función creciente, el θ que maximiza $\ell(\theta)$ es el mismo que maximiza $L(\theta)$. Además del motivo algebraico, hay un motivo numérico: un producto de muchas probabilidades menores que 1 se vuelve rápidamente un número tan pequeño que la computadora lo redondea a 0.

3.2. Lo que aporta cada ejemplo

Como y vale 0 o 1, en cada término de $\ell(\theta)$ solo uno de los dos logaritmos está activo:

$$\text{aporte del ejemplo} = \begin{cases} \log h_\theta(\mathbf{x}) & \text{si } y = 1 \\ \log(1 - h_\theta(\mathbf{x})) & \text{si } y = 0. \end{cases}$$

Estos valores siempre son menores o iguales a cero, porque son logaritmos de probabilidades. Así que $\ell(\theta) \leq 0$, y el mejor valor posible sería $\ell(\theta) = 0$, que corresponde a asignarle probabilidad 1 a todas las etiquetas observadas.

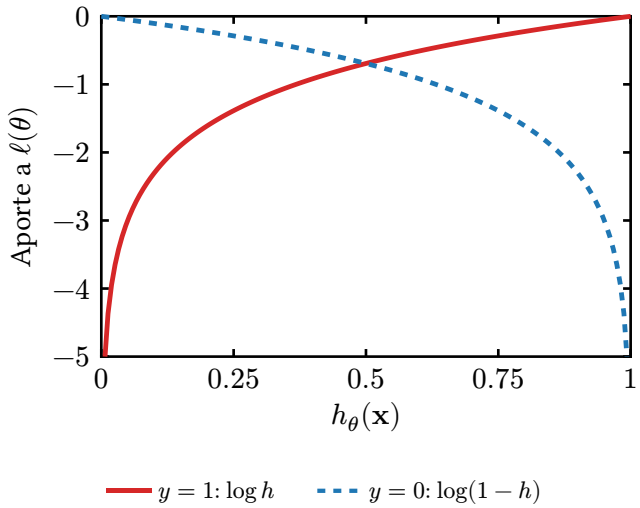


Figura 5: Aporte de un ejemplo a la log-verosimilitud en función de la probabilidad h que el modelo le asigna a la clase 1. Si $y = 1$ (rojo), el aporte es $\log h$: vale cero cuando $h = 1$ y se va a $-\infty$ cuando $h \to 0$. Si $y = 0$ (azul), el aporte es $\log(1 - h)$, con el comportamiento opuesto.

La Figura 5 muestra el comportamiento. Si el ejemplo es de la clase 1 y el modelo le asigna probabilidad 0.9, su aporte es $\log 0.9 \approx -0.11$. Si le asigna 0.1, su aporte es $\log 0.1 \approx -2.3$. Y si le asigna una probabilidad cercana a cero, el aporte es enorme y negativo. Para maximizar $\ell(\theta)$, el modelo tiene que evitar sobre todo los errores hechos con mucha seguridad.

Esto también resuelve el problema de la Figura 1. Un tumor muy grande y maligno tiene un score muy positivo, $h_\theta \approx 1$, y su aporte es prácticamente cero, que es lo máximo posible. No tira de la frontera como lo hacía el error cuadrático.

4. Parte 4: Entrenamiento

4.1. El gradiente

Como queremos maximizar $\ell(\theta)$, vamos a usar ascenso por gradiente: en lugar de restar el gradiente, lo sumamos, porque el gradiente apunta en la dirección en la que la función crece más rápido:

$$\theta := \theta + \alpha \nabla_\theta \ell(\theta).$$

Calculemos la derivada para un solo ejemplo $(\mathbf{x}, y)$. Escribimos $h = g(\theta^T \mathbf{x})$ y usamos la regla de la cadena junto con $g'(z) = g(z)(1 - g(z))$:

$$\begin{aligned}
\frac{\partial}{\partial \theta_j}[y \log h + (1-y)\log(1-h)] &= \left(\frac{y}{h} - \frac{1-y}{1-h}\right)\frac{\partial}{\partial \theta_j} g(\theta^T \mathbf{x}) \\
&= \left(\frac{y}{h} - \frac{1-y}{1-h}\right) h(1-h) x_j \\
&= (y(1-h) - (1-y)h) x_j \\
&= (y - h) x_j.
\end{aligned}$$

En el segundo paso usamos que $\partial(\theta^T \mathbf{x})/\partial \theta_j = x_j$. Sumando sobre todos los ejemplos:

$$\frac{\partial \ell(\theta)}{\partial \theta_j} = \sum_{i=1}^{m} \big(y^{(i)} - h_\theta(\mathbf{x}^{(i)})\big) x_j^{(i)}.$$

En forma matricial, si $\mathbf{h}$ es el vector con las predicciones $h_\theta(\mathbf{x}^{(i)})$ de todos los ejemplos,

$$\nabla_\theta \ell(\theta) = \mathbf{X}^T (\mathbf{y} - \mathbf{h}).$$

Cada ejemplo empuja a θ en la dirección de $\mathbf{x}^{(i)}$, con una fuerza proporcional a su error $y^{(i)} - h_\theta(\mathbf{x}^{(i)})$. Un ejemplo bien clasificado con mucha seguridad tiene un error cercano a cero y casi no influye.

4.2. Ascenso por gradiente

Sustituyendo en la regla de actualización:

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \big(y^{(i)} - h_\theta(\mathbf{x}^{(i)})\big) x_j^{(i)}, \quad j = 0, \ldots, n-1.$$

Y la versión estocástica, que usa un solo ejemplo a la vez:

$$\theta_j := \theta_j + \alpha \big(y^{(i)} - h_\theta(\mathbf{x}^{(i)})\big) x_j^{(i)}.$$

Si comparamos con la regla LMS de regresión lineal, $\theta_j := \theta_j + \alpha\big(y^{(i)} - h_\theta(\mathbf{x}^{(i)})\big)x_j^{(i)}$, vemos que es idéntica. Pero no es el mismo algoritmo: en regresión lineal $h_\theta(\mathbf{x}) = \theta^T \mathbf{x}$, y aquí $h_\theta(\mathbf{x}) = g(\theta^T \mathbf{x})$. Es un resultado algo sorprendente, ya que llegamos a la misma regla partiendo de suposiciones muy distintas.

Todo lo que vimos sobre descenso por gradiente se aplica igual al ascenso: la versión batch usa todos los ejemplos en cada paso, SGD usa uno a la vez, y mini-batch usa pequeños grupos. También ayuda estandarizar las features.

A diferencia de la regresión lineal, ahora no hay solución cerrada. Al igualar el gradiente a cero, θ queda adentro de la sigmoide y no se puede despejar. Por eso siempre se usa un método iterativo.

La buena noticia es que $\ell(\theta)$ es cóncava, así que no tiene máximos locales: si el ascenso por gradiente converge, llega al máximo global.

4.3. Datos separables y regularización

Hay un caso donde el ascenso por gradiente no converge. Supongamos que los datos son linealmente separables y que θ los separa. Como vimos en la Parte 1, si multiplicamos θ por $c > 1$ la frontera no cambia, pero todos los scores se multiplican por c. Los ejemplos de la clase 1 tienen scores aún más positivos y los de la clase 0 aún más negativos, así que todas las probabilidades se acercan más a la

etiqueta correcta y ℓ aumenta. Esto pasa para cualquier c, así que el máximo no existe: ℓ se acerca a 0 a medida que $\|\theta\|$ crece sin límite, pero nunca llega.

En la práctica esto significa que, con datos separables, el ascenso por gradiente hace que los parámetros crezcan indefinidamente, y el modelo termina asignando probabilidades de casi exactamente 0 o 1. Eso es una forma de overfitting: el modelo está demasiado seguro.

La solución es la misma que en regresión lineal: penalizar el tamaño de los parámetros. Con regularización L_2 maximizamos:

$$\max_{\theta} \ell(\theta) - \frac{\lambda}{2} \sum_{j=1}^{n-1} \theta_j^2,$$

y la regla de actualización gana el término $-\alpha\lambda\theta_j$ para $j \geq 1$. Ahora el crecimiento de θ tiene un costo, y el máximo existe siempre. También se puede usar regularización L_1 para obtener modelos sparse. Igual que antes, se escoge λ con validación y se estandarizan las features.

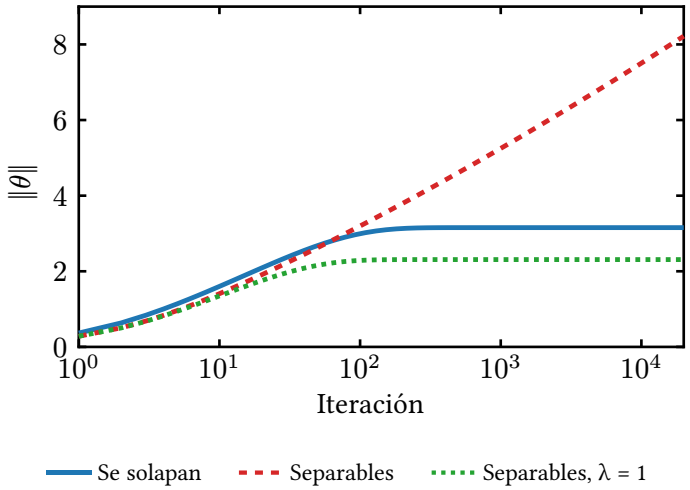


Figura 6: Norma de los parámetros $\sqrt{\theta_1^2 + \theta_2^2}$ durante el ascenso por gradiente, en escala logarítmica para las iteraciones. Con datos que se solapan (azul), la norma se estabiliza. Con datos separables (rojo), sigue creciendo sin detenerse. Con los mismos datos separables y regularización L_2 con $\lambda = 1$ (verde), vuelve a estabilizarse.

Adventercia: `LogisticRegression` de scikit-learn regulariza por defecto, y no usa λ sino un parámetro `C` que es su inverso: con la convención de estas notas, `C` $= 1/\lambda$. Un `C` pequeño significa mucha regularización.

5. Parte 5: Transformaciones de features

5.1. Fronteras no lineales con un modelo lineal

Hasta ahora, la frontera de decisión siempre es un hiperplano. Pero hay problemas donde ninguna recta sirve. En la Figura 7, los ejemplos de la clase 1 están por encima de una parábola y los de la clase 0 por debajo, a ambos lados. Si entrenamos una regresión logística con x_1 y x_2, la mejor recta acierta en el 79% de los ejemplos: no puede dejar de un mismo lado los dos grupos de la clase 0.

La solución es la misma que usamos para la regresión polinomial: construir nuevas features a partir de las originales. Definimos una transformación, o feature map,

$$\phi : \mathbb{R}^d \to \mathbb{R}^p,$$

que toma las d variables originales y produce p features nuevas. El modelo es ahora

$$h_\theta(\mathbf{x}) = g(\theta^T \phi(\mathbf{x})),$$

con $\theta \in \mathbb{R}^p$. Para el ejemplo de la figura usamos

$$\phi(\mathbf{x}) = \begin{pmatrix} 1 \\ x_1^2 \\ x_2 \end{pmatrix}.$$

La frontera de decisión $\theta_0 + \theta_1 x_1^2 + \theta_2 x_2 = 0$ es una recta en el espacio de las features (x_1^2, x_2), pero en el espacio original es la parábola $x_2 = -(\theta_0 + \theta_1 x_1^2)/\theta_2$. Con esta transformación el modelo acierta en el 96% de los ejemplos.

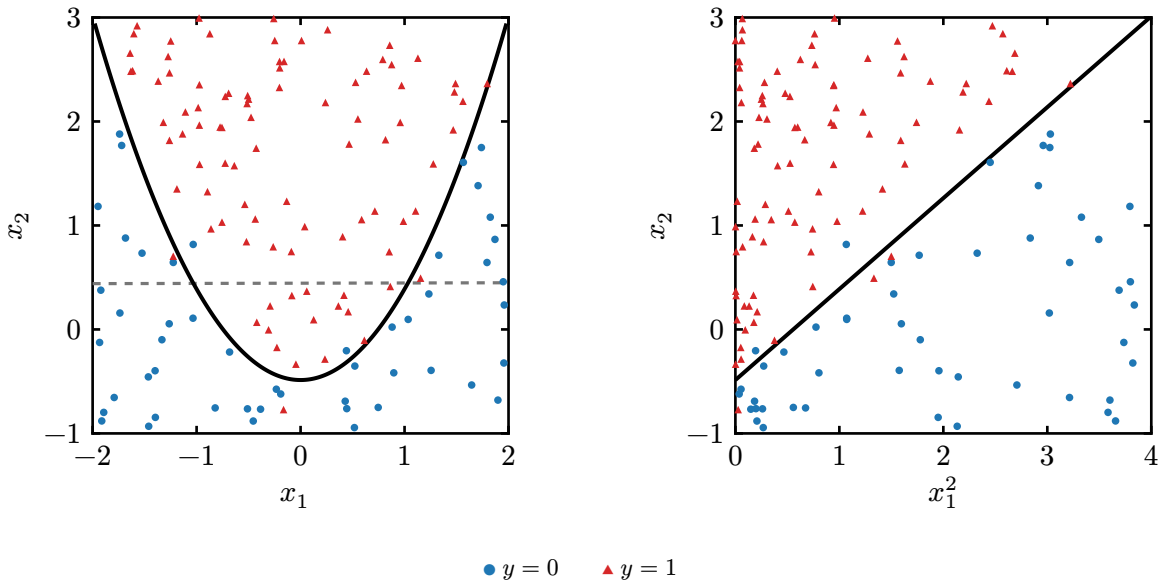


Figura 7: A la izquierda, datos en el espacio original. La recta punteada es la frontera de la regresión logística con (x_1, x_2); la curva continua es la frontera del modelo entrenado con $\phi(\mathbf{x}) = (1, x_1^2, x_2)$. A la derecha, los mismos datos en el espacio transformado (x_1^2, x_2), donde esa misma frontera es una recta.

Nada del entrenamiento cambia: basta reemplazar $\mathbf{x}^{(i)}$ por $\phi(\mathbf{x}^{(i)})$ en todas las fórmulas. Por ejemplo, la regla de ascenso por gradiente queda

$$\theta := \theta + \alpha \sum_{i=1}^{m} \left(y^{(i)} - g(\theta^T \phi(\mathbf{x}^{(i)}))\right) \phi(\mathbf{x}^{(i)}).$$

El modelo sigue siendo lineal en θ, y por eso sigue siendo fácil de entrenar. Lo no lineal está en ϕ, que escogemos nosotros.

Las mismas advertencias de la regresión polinomial aplican aquí. Con un ϕ muy rico (por ejemplo, todos los monomios hasta grado 10), la frontera puede volverse tan flexible que rodee cada ejemplo de entrenamiento: overfitting. La complejidad se controla igual que antes, escogiendo ϕ con validación y usando regularización.

5.2. El costo de muchas features

En el ejemplo anterior sabíamos qué transformación usar. Muchas veces no lo sabemos, y la opción natural es incluir todos los términos cuadráticos. Si $\mathbf{x}$ tiene d variables:

$$\phi(\mathbf{x}) = (1, x_1, \ldots, x_d, x_1x_1, x_1x_2, \ldots, x_1x_d, x_2x_1, \ldots, x_dx_d).$$

Si contamos cada producto en sus dos órdenes (por ejemplo, x_1x_2 y x_2x_1 por separado), hay $1 + d + d^2$ features. Con $d = 10000$, eso es alrededor de 10^8 features, y cada paso del ascenso por gradiente requiere calcular y guardar vectores de ese tamaño. Con términos de grado 3 el número crece como d^3. Parece que transformar las features nos obliga a pagar un costo enorme.

5.3. Hacia SVM y kernels

Hay una observación que cambia el panorama. Supongamos que solo necesitamos productos internos entre features, $\phi(\mathbf{x})^T\phi(\mathbf{z})$. Para el ϕ cuadrático:

$$\phi(\mathbf{x})^T\phi(\mathbf{z}) = 1 + \sum_{i=1}^{d} x_i z_i + \sum_{i=1}^{d}\sum_{j=1}^{d} x_i x_j z_i z_j.$$

El último término se puede reorganizar:

$$\sum_{i=1}^{d}\sum_{j=1}^{d} x_i x_j z_i z_j = \left(\sum_{i=1}^{d} x_i z_i\right)\left(\sum_{j=1}^{d} x_j z_j\right) = \left(\mathbf{x}^T\mathbf{z}\right)^2.$$

Entonces

$$\phi(\mathbf{x})^T\phi(\mathbf{z}) = 1 + \mathbf{x}^T\mathbf{z} + \left(\mathbf{x}^T\mathbf{z}\right)^2.$$

El lado izquierdo es un producto interno entre vectores de 10^8 componentes. El lado derecho solo requiere calcular $\mathbf{x}^T\mathbf{z}$, que tiene $d = 10000$ términos, y luego unas pocas operaciones. A la función $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^T\phi(\mathbf{z})$ se le llama kernel.

¿Y cuándo necesitamos solo productos internos? Miremos la regla de ascenso por gradiente con features. Si empezamos con $\theta = 0$, cada paso le suma a θ una combinación de los vectores $\phi\big(\mathbf{x}^{(i)}\big)$. Por lo tanto, en cualquier momento $\theta = \sum_i \beta_i \phi\big(\mathbf{x}^{(i)}\big)$ para ciertos números β_i, y el score de un punto nuevo es

$$\theta^T\phi(\mathbf{x}) = \sum_{i=1}^{m} \beta_i \phi\big(\mathbf{x}^{(i)}\big)^T\phi(\mathbf{x}) = \sum_{i=1}^{m} \beta_i K\big(\mathbf{x}^{(i)}, \mathbf{x}\big).$$

Es decir, podríamos entrenar y predecir sin construir nunca ϕ: basta con guardar los m números β_i y saber calcular K. Esta idea se conoce como el kernel trick, y la desarrollaremos en detalle más adelante.

Para cerrar, dos preguntas que quedan abiertas y que responden las máquinas de soporte vectorial:

1. Cuando los datos son separables, ¿cuál de todas las fronteras posibles es la mejor? La respuesta será la frontera con mayor margen, es decir, la que queda lo más lejos posible de los puntos más cercanos de cada clase.
2. ¿Se puede escribir ese problema de forma que solo dependa de productos internos, para usar kernels y obtener fronteras no lineales sin calcular ϕ?

En ese contexto es costumbre separar el intercepto del resto de los parámetros. Si llamamos $b = \theta_0$ y $\mathbf{w} = (\theta_1, \ldots, \theta_{n-1})$, el score se escribe

$$\theta^T\mathbf{x} = \mathbf{w}^T\mathbf{x} + b,$$

donde ahora $\mathbf{x}$ no incluye el 1 inicial. El vector $\mathbf{w}$ es el vector normal a la frontera, y la distancia de un punto a la frontera es $(\mathbf{w}^T\mathbf{x} + b)/\|\mathbf{w}\|$. Esta es la notación que usaremos con SVM.

6. Resumen

- Un clasificador lineal predice con el signo del score $\theta^T \mathbf{x}$. Su frontera de decisión es el hiperplano $\theta^T \mathbf{x} = 0$, con vector normal $(\theta_1, ..., \theta_{n-1})$.
- La regresión logística pasa el score por la sigmoide, $h_\theta(\mathbf{x}) = g(\theta^T \mathbf{x})$, y lo interpreta como la probabilidad de que $y = 1$.
- Los parámetros se escogen por máxima verosimilitud, maximizando $\ell(\theta) = \sum y \log h_\theta + (1 - y) \log(1 - h_\theta)$.
- El gradiente es $\sum \big(y^{(i)} - h_\theta\big(\mathbf{x}^{(i)}\big)\big)\mathbf{x}^{(i)}$, y la regla de ascenso por gradiente tiene la misma forma que la regla LMS. No hay solución cerrada, pero ℓ es cóncava y el ascenso por gradiente encuentra el máximo global.
- Con datos separables, los parámetros crecen sin límite; la regularización lo evita.
- Con un feature map ϕ obtenemos fronteras no lineales sin dejar de tener un modelo lineal en θ. Cuando ϕ tiene muchas componentes, los kernels permiten calcular productos internos $\phi(\mathbf{x})^T \phi(\mathbf{z})$ sin construir ϕ.